

MTC Sim-2.0

*\$ACM membership, Meme Token Credit settlement,
and the Walmart-week calendar everything hangs from*

Version 2.0 simulation · August 2026 · ~/Air/aladdins-castle/mtc

Prepared for T. Wichmann · CONFIDENTIAL

NOTHING IN THIS BUILD IS REAL. No blockchain writes. No funded wallets. No physical scanners. Wallet addresses are marked `sim:true` and cannot be funded. Brand names are TEST FIXTURES chosen to make the flow legible — no relationship, agreement or endorsement is implied or exists.

01

What was built, in one paragraph

A consumer enrolls and we mint them a Solana wallet whose key we hold; one \$ACM lands in it and that token is their membership credential, nothing more. A brand funds an offer denominated in Walmart weeks. We push that consumer a **Meme Token Credit** — which is not a coin but an Apple Wallet pass carrying an HMAC-signed QR code. They walk into an arcade, the reader scans it, and that scan is the settlement event: the credit is extinguished exactly once, the product is handed over, and a row lands in an append-only hash-chained ledger. At the close of each Walmart week a Merkle root of that ledger is published to a public chain, so a brand can verify its own redemptions against a commitment nobody — including us — can revise.

The blockchain does exactly one job in this system: prove the holder has at least one \$ACM. That is a read. Everything else is our ledger.

02

The Walmart-week spine

Every offer window, expiry and ledger period is denominated in Walmart weeks. This is not cosmetic. Walmart's trade team plans, funds and reports promotional dollars in these weeks, so a redemption ledger kept in the same weeks reconciles to a brand's own trade report with no translation step — no partial weeks, no allocation assumptions, no argument about which week a Tuesday redemption belongs to.

Rule	Implementation
Week boundary	Saturday 00:00:00 → Friday 23:59:59
Timezone	America/New_York
Fiscal year	Feb 1 → Jan 31, opening on the Saturday of the week containing Feb 1
Week numbering	Week 1 is Walmart week 1 — no parallel numbering
53-week years	Derived from the calendar, never assumed or tabled
Offer duration	1 week minimum, 52 weeks maximum; rolls into the next FY if it must

Why America/New_York and not EST

Strict "EST" is UTC-5 year-round. From March to November an offer set to expire Friday midnight would actually die at 1:00 a.m. local — an hour the customer believes they still have. The named zone observes the DST shift, so Friday midnight is midnight on the wall clock every week of the year.

The 53-week year, handled without a special case

```
# A fiscal year runs until the next one starts. The week count falls out
# of the two anchors, so a 53-week year needs no special case anywhere else.
def fy_start(fiscal_year):
    anchor = date(fiscal_year - 1, 2, 1)          # FY2027 -> Feb 2026
    return anchor - timedelta(days=(anchor.weekday() - SATURDAY) % 7)

def fy_weeks(fiscal_year):
    return (fy_start(fiscal_year + 1) - fy_start(fiscal_year)).days // 7
```

```
FY2027: starts 2026-01-31 (Sat), 52 weeks
FY2028: starts 2027-01-30 (Sat), 52 weeks
FY2031: starts 2030-01-26 (Sat), 53 weeks    <- derived, not hardcoded
```

03

HMAC on every channel

QR payloads, reader-to-server reports, brand submissions, webhooks, internal calls. One standard, no per-case decision.

A random token requires the verifier to ask a server whether it is real. An HMAC-signed token verifies OFFLINE, with no network at all.

That is load-bearing: Starlink is the arcade site minimum, but a scanner that stops working during an outage stops the entire promotion. Signed codes mean the worst case is delayed reconciliation, not a dead lane.

Three things that are not optional

Element	Why it must be there
Key ID in the payload	Readers are deployed in the field. Without it you cannot rotate a secret without bricking every one of them.
Timestamp + nonce inside the signed envelope	Without them a captured valid signature stays valid forever and can simply be replayed.
Constant-time compare	Comparing signatures with == leaks how many leading bytes matched, through timing. That is a documented path to forging a signature one byte at a time.

```
# A nonce and issued-at are injected HERE, not trusted from the caller, so no
# call site can forget them and quietly create a replayable token.
def sign(payload, key, key_id):
    body = dict(payload)
    body.setdefault("iat", int(time.time()))
    body.setdefault("jti", secrets.token_hex(8))
    raw = _b64(json.dumps(body, separators=(",", ":"), sort_keys=True).encode())
    sig = _b64(hmac.new(_unb64(key), f"{key_id}.{raw}".encode(),
                        hashlib.sha256).digest())
    return f"{key_id}.{raw}.{sig}"

def verify(token, keyring, max_age_s=None):
    kid, raw, sig = token.split(".")
    ...
    if not hmac.compare_digest(sig, expect):      # constant time – never ==
        return False, "bad signature"
```

```
# measured on a live token
token length      : 168–395 chars (fits a QR comfortably)
clean verify      : True
tampered payload  : (False, 'bad signature')
wrong key         : (False, 'bad signature')
unknown key id    : (False, 'unknown key id rdr-999')
```

04

The MTC is not a coin

Two different things were both called "wallet," and that is what makes this confusing on first reading:

Thing	Holds	Touches the chain?
SOL wallet	1 \$ACM — eligibility	Read only. Never spent.
Apple Wallet	The MTC pass — the offer	Never.

What that choice buys

Operation	Off-chain pass (built)	On-chain token (rejected)
Issue a credit	a database write	a transaction + fee
Expire 10,000 credits	one timestamp query, free	10,000 burn transactions
Non-transferable	true by construction	needs a token extension
User needs SOL for gas	no	yes
"Extinguished exactly once"	first valid scan wins a race	a burn transaction

05

Wallet minting — the noob path

Solana does not mine keys. Bitcoin's proof-of-work orders *transactions*; it has nothing to do with creating a key. A Solana private key is literally 32 random bytes, and the public address is derived from it mathematically, offline, in microseconds. There is nothing to search for and no network call involved in creating one.

1. 32 bytes from the OS CSPRNG (SecRandomCopyBytes on macOS)
2. derive the ed25519 public key -> that IS the address, offline
3. confirm the address has never been used
4. push 1 \$ACM -> the eligibility credential

Step 3 is not about collisions. Colliding with an existing wallet is 1 in 2^{256} . It is about catching a broken RNG on our side — and a bug in our own randomness is nowhere near 1 in 2^{256} . One cheap call turns a silent catastrophic failure into a loud one.

Two things flagged, not solved

Custody. Holding the key on the user's behalf is custody, with the regulatory posture that implies. The architecture is positioned for it — MTC carries no cash value and \$ACM is never spendable inside the system — but "we can revoke it" and "the user owns it" are different claims and only one can be true. This is a specific question for Wood Herron, not a footnote.

Key storage at scale. The simulation writes keys to a local file. That is fine for a simulation and wrong for production: N independent keys means N secrets to protect and one breach exposes everyone. Production wants hierarchical deterministic derivation — one master seed in an HSM, each user's key derived by index — so there is one secret to guard and no database of private keys exists to steal.

06

The ledger is the product

KRW-C-2026-002: "The measurement is not a product built alongside the system. It is what the system leaves behind." Every state change goes through the ledger. There is no other write path, and the ledger has no update and no delete.

```
# Hash over every field INCLUDING the previous hash. Order is part of the
# contract — change it and every historical hash becomes unverifiable.
def _hash_row(seq, utc, fy, week, event, subject, detail, prev_hash):
    blob = json.dumps([seq, utc, fy, week, event, subject, detail, prev_hash],
                      separators=(",", ":"), sort_keys=True)
    return hashlib.sha256(blob.encode()).hexdigest()
```

Altering any historical row changes every hash after it, and the break is detectable by anyone holding a later hash. Verified on a tampered file:

```
clean chain      : (True, None, 5)
after tampering : (False, 3, 5)    <- detected at that sequence number
```

Why git alone is not enough

Git is hash-chained and therefore genuinely tamper-evident. But history can be force-pushed and rewritten by anyone with write access, and nothing inside the repository would prove it happened. So a Merkle root is published per Walmart week:

The repository holds the data. The chain holds the proof that the data has not moved. A brand verifies its own redemptions against an anchor nobody controls — including us.

Scoping the anchor to a week is deliberate: the root published at week close commits exactly that week's settlement facts, which is the period the brand's trade report is denominated in.

07

Redemption — the exactly-once guarantee

```
# THE PATENT CLAIM, in five lines. First valid scan wins; every later scan
# of the same instrument fails here, forever.
if p["state"] == "REDEEMED":
    self.ledger.append("REDEEM_REJECTED", mid,
                      {"reason": "already redeemed", "reader": reader_id})
    return {"ok": False, "reason": "already redeemed"}
p["state"] = "REDEEMED"
```

The offline tradeoff, stated rather than discovered

Offline validation proves a code is authentic and unexpired with no network. It *cannot* check whether the code was already spent — that needs the ledger. So an offline scan is queued and reconciled when the link returns, accepting a small double-redemption window rather than letting an outage stop the lane. That is a business decision and it should be made deliberately, not found later.

08

First run

SIM-2.0 · FY2027 WK27 · Sat 01 Aug — Fri 07 Aug 2026

140 members enrolled	140 \$ACM out of float
332 MTC passes issued	139 redeemed
623 ledger events	chain intact: True

PG-TIDE	WK27 + 1wk	→ FY2027 WK27	100 issued	45 redeemed	\$67.50
PG-DAWN	WK27 + 2wk	→ FY2027 WK28	83 issued	34 redeemed	\$34.00
PG-CHRM	WK27 + 4wk	→ FY2027 WK30	66 issued	26 redeemed	\$52.00
PG-GILL	WK27 + 13wk	→ FY2027 WK39	50 issued	23 redeemed	\$69.00
PG-OLAY	WK27 + 52wk	→ FY2028 WK26	33 issued	11 redeemed	\$27.50

ADVERSARIAL SCANS

duplicate scan	→ already redeemed
tampered QR	→ bad signature
unknown reader key	→ unknown key id ARC-FAKE-99
OFFLINE VALIDATION	→ authentic + unexpired, NO network

The simulation caught a bug on its first run — an invented rule

An earlier draft refused any offer running past fiscal year end. That silently made a 52-week offer *impossible* unless it began in week 1, and PG-OLAY failed immediately. The stated rule is **duration**: 1 to 52 weeks from wherever it starts. The window now rolls into the next fiscal year and reports the closing week in the year it actually lands in — which is why PG-OLAY reads WK27 + 52 → FY2028 WK26.

09

Enrollment economics, measured

Item	Cost per user
Associated Token Account rent	0.00204 SOL $\approx$ \$0.408 (recoverable on close)
Transfer fee	$\approx$ \$0.001
The 1 \$ACM itself	\$0.000265
Total	$\approx$ \$0.41 , almost all of it recoverable rent

At 100,000 enrolments that is **\$40,927**, of which **\$40,800 is rent** locked while accounts are open — a working-capital lockup rather than a cost, and worth knowing before it surprises anyone at scale.

At 1 \$ACM per user the cooling mechanism of KRW-C-2026-002 does not move. One million enrolled customers removes 0.1% of circulating supply.

Circulating supply is 999,957,146. The paper hedges this correctly — "*the vaulting rate per enrollment is illustrative and not a committed program parameter*" — but the parameter has to be set, and 1 is the wrong number. At current price, 10,000 ACM costs **\$2.65**; vault that per user and 10,000 enrolments removes **10% of circulating supply**. Eligibility threshold and vaulting quantity are separate parameters, and only the second one cools anything.

10

What exists, and what does not

Component	State	Note
Walmart-week engine	BUILT	53-week years derived; verified against Walmart supplier documentation
HMAC auth, all channels	BUILT	key id, nonce, constant-time compare
Hash-chained ledger	BUILT	tamper detection verified on a modified file
MTC pass + offline validation	BUILT	QR fits, validates with no network
Exactly-once redemption	BUILT	duplicate, forged and unknown-key scans all rejected
Monitor	BUILT	http://127.0.0.1:8901
Merkle anchoring	SIMULATED	root computed correctly; chain write is a stub
Real ed25519 derivation	SIMULATED	addresses marked sim:true; one-line change to go real
Key storage	NOT PRODUCTION	local file. Needs HD derivation from an HSM seed
Apple Wallet pass generation	NOT BUILT	needs an Apple Pass Type ID certificate
Physical reader	NOT BUILT	iPhone SE scanner app